Accelerated Mobile Video Processing System

Real-Time Camera Pipeline using SIMD and Mobile GPU

1 Problem Statement

The objective of this project is to design and build a **real-time camera-based video processing application on Android** that achieves measurable acceleration through the coordinated use of **CPU vector execution (SIMD)** and the **mobile GPU**.

The application must process live camera frames continuously and display the transformed output in real time. The system must demonstrate clear performance gains relative to a non-accelerated baseline while preserving functional correctness.

This project is evaluated as a **systems and performance engineering problem**, not as a graphics or UI exercise.

2 System Scope and End-to-End Requirement

The complete system must satisfy the following end-to-end requirement:

Live Camera Input → Accelerated Frame Processing → On-Screen Display

Frames must be captured, processed, and rendered continuously without manual triggering or offline pre-processing.

All computation must occur **entirely on the mobile device**. Use of external servers or cloud-based computation is not permitted.

3 Application Definition: Real-Time Camera Video Processing

The application operates as a live camera pipeline that applies a compute-intensive transformation to each incoming frame before display.

The specific transformation is left open-ended, provided it satisfies the following constraints:

- Operates on every pixel or local neighborhood of pixels
- Executes continuously on streaming camera frames
- Has a well-defined notion of correctness
- Is computationally significant enough to benefit from acceleration

The system must maintain interactive frame rates under sustained operation.

4 Accelerated Execution Model

The application must support multiple execution modes:

1. **Baseline Mode**: Scalar CPU execution
2. **SIMD-Accelerated Mode**: Vectorized CPU execution
3. **GPU-Accelerated Mode**: Mobile GPU execution
4. **Hybrid Mode**: Coordinated use of SIMD and GPU

The output produced in all modes must be functionally equivalent within acceptable numerical tolerance.

Switching between execution modes must occur at runtime without restarting the application.

5 Central Role of the Performance Dashboard

A real-time performance dashboard is a **mandatory deliverable** and is considered a core system component.

The dashboard must be visible during execution and update live as frames are processed.

Minimum Dashboard Requirements

The dashboard must display:

- Current execution mode (baseline, SIMD, GPU, hybrid)
- Live camera frame rate (FPS)
- Per-frame processing latency
- End-to-end latency (camera capture to display)
- SIMD utilization indicator
- GPU execution activity indicator
- Thermal or power proxy indicators (where available)

The dashboard should allow an observer to reason about **how computation is mapped onto hardware resources** in real time.

6 80-Day Development Plan and Partial Deliverables

The project is divided into five phases, each producing a demonstrable system artifact.

6.1 Day 20: Camera Pipeline and Baseline System

Deliverables:

- Live camera capture integrated into the application
- Continuous frame flow from camera to display
- Baseline (non-accelerated) frame processing
- Dashboard showing frame rate and baseline latency

6.2 Day 40: SIMD-Accelerated Frame Processing

Deliverables:

- SIMD-accelerated execution path integrated into the pipeline
- Verified output correctness relative to baseline
- Dashboard reporting SIMD execution latency
- Measured performance improvement over baseline

6.3 Day 70: GPU-Accelerated Frame Processing

Deliverables:

- GPU-accelerated execution path integrated into the pipeline
- Verified output correctness relative to baseline
- Dashboard reporting GPU execution latency
- Measured performance improvement over baseline
- Coordinated use of SIMD and GPU within the same frame pipeline
- Runtime execution mode switching
- Dashboard showing per-resource contribution
- Stable operation under sustained load

6.4 Day 80: End-to-End Optimization and Finalization

Deliverables:

- Fully integrated real-time camera processing system
- Sustained operation at interactive frame rates
- Unified end-to-end latency measurement
- Dashboard suitable for live demonstration

—

7 Final Demonstration Requirements

The final demonstration must satisfy:

- Execution on a physical Android phone
- Fully offline operation
- Live camera input and continuous output display
- Real-time dashboard visible during execution
- Quantitative performance improvement over baseline
- No manual intervention during runtime

The system must operate stably for an extended demonstration period.

—

8 Evaluation Criteria

The system will be evaluated on:

- Functional correctness of video output
- Achieved performance acceleration
- End-to-end latency and frame stability
- Quality of SIMD and GPU utilization
- Clarity and usefulness of the dashboard
- Overall system integration quality

Superficial or non-observable acceleration will not receive full credit.
—

9 Expected Outcome

By the end of the project, the system should convincingly demonstrate how mobile heterogeneous compute resources can be exploited to accelerate real-time workloads.

The final artifact should be suitable for live demonstration, technical evaluation, and extension toward power-aware or adaptive mobile computing research.